

Here is a **DevOps interview–focused, detailed yet structured guide** on **GitHub Actions Expressions & Contexts** (copy-paste + revision friendly).

✓ 1. What are Expressions in GitHub Actions

📖 Definition

Expressions are used to **evaluate values dynamically** inside workflow YAML using:

```
${{ expression }}
```

✓ Where Expressions are Used

```
if: ${{ condition }}
```

```
env:
```

```
  VAR: ${{ value }}
```

```
with:
```

```
  param: ${{ value }}
```

```
run: echo "${{ github.ref }}"
```

✓ Core Features

- ✓ Access **context values**
 - ✓ Perform **conditions**
 - ✓ Do **string manipulation**
 - ✓ Use **functions**
-

✓ 2. Expression Syntax Basics

✓ Example

```
if: ${{ github.ref == 'refs/heads/main' }}
```

✓ Supported Operators

Type	Operators	
Comparison	==, !=, >, <, >=, <=	
Logical	&&, `	, !`
Others	contains, startsWith, endsWith	

✓ Example

```
if: ${ { github.event_name == 'push' && github.ref == 'refs/heads/main' } }
```

✓ 3. Functions in Expressions (Must Know for Interview)

✓ 1. contains()

```
if: ${ { contains(github.ref, 'main') } }
```

🔗 Check substring

✓ 2. startsWith()

```
if: ${ { startsWith(github.ref, 'refs/tags/') } }
```

✓ 3. endsWith()

```
if: ${ { endsWith(github.ref, 'release') } }
```

✓ 4. format()

```
${ { format('Hello {0}', github.actor) } }
```

✓ 5. join()

```
${ { join(github.event.labels.*.name, ', ') } }
```

✓ 6. toJson() (debugging)

run: echo '\${{ toJson(github) }}'

``

✓ 7. fromJson()

\${{ fromJson('["dev","prod"]')[0] }}

``

✓ 8. hashFiles()

\${{ hashFiles('**/package-lock.json') }}

``

👉 Used in **caching**

✓ 4. What are Contexts in GitHub Actions

👉 Definition

Contexts are **predefined objects** that provide runtime information.

✓ Syntax

\${{ context.property }}

``

✓ 5. Most Important Contexts (INTERVIEW CRITICAL)

✓ 1. github Context (MOST IMPORTANT)

👉 Contains workflow + repo + event info

✓ Common Fields

\${{ github.ref }} # branch/tag

```
${{ github.repository }} # owner/repo
${{ github.actor }}      # username
${{ github.event_name }} # push, pull_request
${{ github.sha }}        # commit ID
${{ github.workspace }}  # runner working dir
```

✓ Example

```
run: echo "Branch is ${{ github.ref }}"
``
```

✓ 2. env Context

👉 Environment variables

env:

```
APP_ENV: prod
```

```
run: echo ${{ env.APP_ENV }}
```

✓ 3. job Context

```
${{ job.status }} # success/failure
```

✓ 4. steps Context

👉 Access outputs of previous steps

```
${{ steps.step_id.outputs.var }}
```

```
``
```

✓ Example

- name: Set output

id: step1

run: echo "version=1.0" >> \$GITHUB_OUTPUT

- name: Use output

run: echo \${ steps.step1.outputs.version }

✓ 5. runner Context

\${ runner.os } # Linux/Windows/Mac

\${ runner.temp }

``

✓ 6. secrets Context (VERY IMPORTANT)

\${ secrets.DOCKER_PASSWORD }

⚠ Used for sensitive data

✓ 7. strategy Context

\${ strategy.matrix.os }

✓ 8. matrix Context

👉 Used in parallel jobs

matrix:

os: [ubuntu-latest, windows-latest]

run: echo \${ matrix.os }

✓ 9. needs Context

👉 Access previous job outputs

```
${{ needs.build.result }}
```

✓ 6. Real DevOps Usage Examples

✓ Example 1: Run only on main branch

```
if: ${{ github.ref == 'refs/heads/main' }}
```

```
``
```

✓ Example 2: Docker login using secrets

```
run: echo "${{ secrets.DOCKER_PASSWORD }}" | docker login -u "${{ secrets.DOCKER_USERNAME }}" --password-stdin
```

```
``
```

✓ Example 3: Matrix build (multi OS)

```
strategy:
```

```
  matrix:
```

```
    os: [ubuntu-latest, windows-latest]
```

```
runs-on: ${{ matrix.os }}
```

✓ Example 4: Conditional deploy

```
if: ${{ github.event_name == 'push' && github.ref == 'refs/heads/main' }}
```

✓ Example 5: Cache dependency

```
- uses: actions/cache@v3
```

```
  with:
```

```
    key: ${{ runner.os }}-${{ hashFiles('**/package-lock.json') }}
```

✓ 7. Important Interview Questions (WITH ANSWERS)

✓ Q1: Expression vs Context?

Expression	Context
Logic evaluator	Data provider
<code>\${{ }}</code> syntax	Predefined objects
Uses functions/operators	Holds values

✓ Q2: How to debug context?

✓ Answer:

run: `echo '${{ toJson(github) }}`

✓ Q3: Difference between env and secrets?

env	secrets
Plain text	Encrypted
Visible in logs	Masked
For normal vars	For passwords/API keys

✓ Q4: How to pass data between steps?

✓ Answer:

`echo "key=value" >> $GITHUB_OUTPUT`

✓ Q5: When is needs used?

👉 For job dependency:

`needs: build`

✓ 8. Common Mistakes (VERY IMPORTANT)

✗ Missing `${{ }}`

if: github.ref == 'main' ✗

✓ Correct:

if: `${{ github.ref == 'refs/heads/main' }}`

✗ Wrong branch name

main ✗

``

✓ Correct:

refs/heads/main

✗ Using secrets incorrectly

echo `$secrets.PASSWORD` ✗

✓ Correct:

echo "`${{ secrets.PASSWORD }}`"

✗ Step ID missing

steps.output ✗

``

✓ Need id

✓ 9. Pro Tips (INTERVIEW IMPRESSION)

- ✓ Use if conditions for **cost optimization**
 - ✓ Use matrix for parallel execution
 - ✓ Use hashFiles for caching
 - ✓ Use needs for job sequencing
 - ✓ Use toJson for debugging
-

✓ 10. One-Line Cheat Sheet

Expressions = logic (\${{
}})

Contexts = data (github, env, secrets, steps, runner, matrix)

If you want, I can give: ✓ 20 real DevOps interview questions on GitHub Actions

- ✓ Full production CI/CD pipeline using expressions
- ✓ A4 printable cheat sheet (your favorite style)

Just tell 👍